



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційні систем та технологій

Лабораторна робота № 8
із дисципліни «Технології розробки програмного забезпечення»
Тема: «Патерни проектування.»

Виконав
Студенти групи ІА-31:
Корнійчук М.Р

Перевірив:
Мягкий М.Ю

Тема: Патерни проектування.

Мета: Вивчити структуру шаблонів «Composite», «Flyweight» (Пристосуванець), «Interpreter», «Visitor» та навчитися застосовувати їх в реалізації програмної системи.

Варіант :

3. Текстовий редактор (strategy, command, observer, template method, flyweight, SOA)

Текстовий редактор повинен вміти розпізнавати текстові файли в будь-якій кодуванні, мати розширені функції редагування: макроси, сніппети, підказки, закладки, перехід на рядок / сторінку, підсвічування синтаксису (для однієї мови програмування або розмітки на розсуд студента).

Посилання на гіт: https://github.com/MkEger/trpz-labs/tree/lab_8

Завдання:

- Ознайомитись з короткими теоретичними відомостями.
- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.
- Реалізувати один з розглянутих шаблонів за обраною темою.
- Реалізувати не менше 3-х класів відповідно до обраної теми.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону.

Хід роботи

Короткий опис виконання роботи:

У рамках виконання лабораторної роботи було розроблено підсистему форматування тексту (підсвічування синтаксису) для проєкту «Текстовий редактор» із використанням структурного патерну Flyweight (Легковаговик). Головною метою

було оптимізувати споживання оперативної пам'яті при роботі з великою кількістю форматуваних елементів. Замість створення окремого об'єкта стилю (Font/Color) для кожного слова, було реалізовано механізм спільного використання (sharing) об'єктів через спеціальну фабрику. Це дозволяє зберігати в пам'яті лише унікальні комбінації стилів, застосовуючи їх до будь-якої кількості слів у тексті.

Тема проекту: Текстовий редактор

1. Реалізувати не менше 3-х класів відповідно до обраної теми

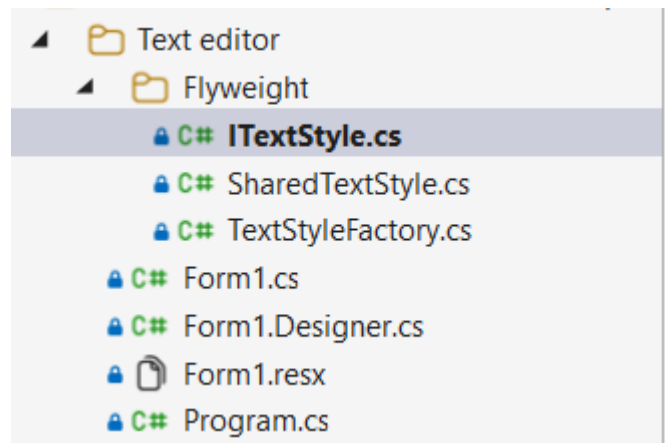


Рис. 1 - Структура проекту

У процесі виконання лабораторної роботи було створено три основні класи, які забезпечують роботу механізму експорту звітів. Наведемо детальний опис кожного класу:

1. ITextStyle (Інтерфейс Flyweight) Загальний інтерфейс для всіх об'єктів-легковаговиків.
 - Apply(RichTextBox control, int index, int length) – метод застосування стилю до тексту. Ключовою особливістю є те, що він приймає зовнішній стан (посилання на контрол та позицію в тексті) як аргументи, не зберігаючи їх усередині об'єкта.
2. SharedTextStyle (Concrete Flyweight) Конкретна реалізація стилю, що зберігає внутрішній стан (intrinsic state).
 - Поля: _font (Шрифт) та _color (Колір). Ці дані є незмінними та спільними для багатьох частин тексту.
 - Метод Apply: Використовує внутрішні дані (шрифт/колір) та зовнішні параметри

(індекс/довжина) для безпосереднього форматування тексту в редакторі.

3. TextStyleFactory (Flyweight Factory) Фабрика, що керує пулом (кешем) легковаговиків.

- GetStyle(...) – метод, який забезпечує повторне використання об'єктів. Він генерує унікальний ключ на основі параметрів шрифту та кольору. Якщо такий стиль вже існує в кеші, повертається посилання на нього; якщо ні — створюється новий екземпляр. Це гарантує, що однакові стилі не дублюються в пам'яті.

2. Реалізувати один з розглянутих шаблонів за обраною темою

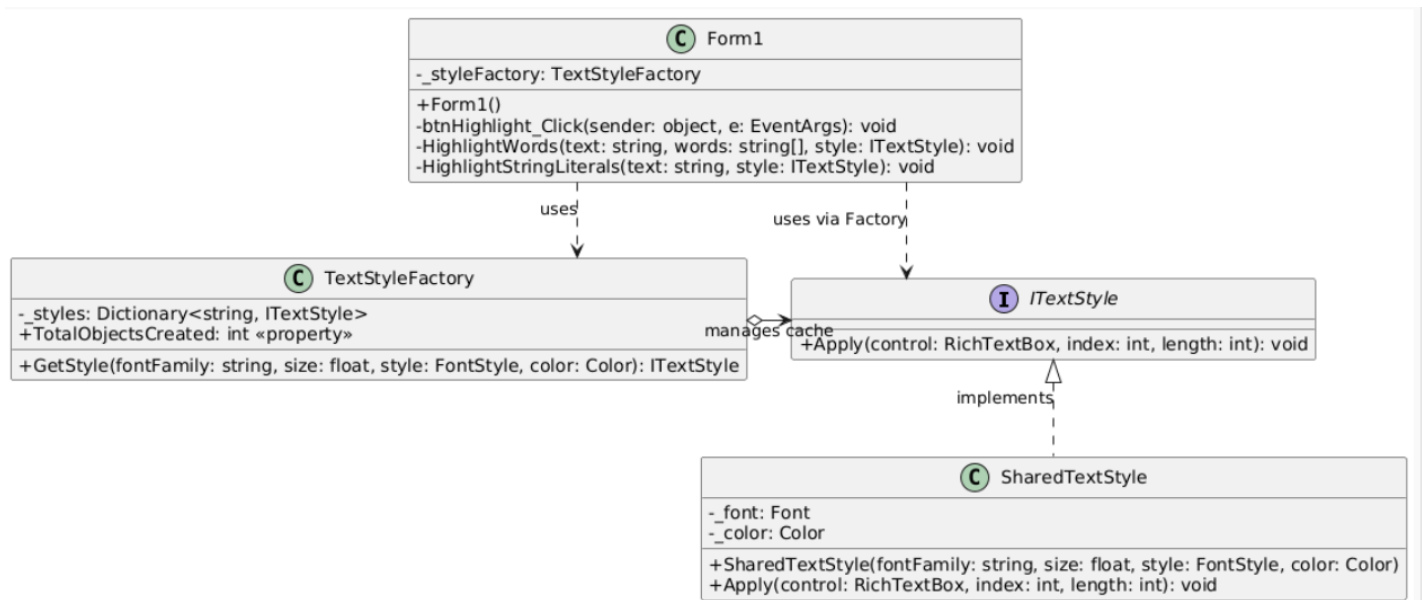


Рис. 2 - Діаграма класів

У рамках проєкту текстового редактора патерн Flyweight було реалізовано для оптимізації підсистеми форматування тексту (Syntax Highlighting). Його використання дозволило значно зменшити споживання оперативної пам'яті при обробці великих документів з повторюваними стилями.

Реалізація: Основна ідея полягала у чіткому розділенні стану об'єктів форматування на внутрішній (шрифт, колір) та зовнішній (позиція в тексті).

- Фабрика (TextStyleFactory) виступає єдиною точкою доступу до стилів, гарантуючи унікальність кожного екземпляра SharedTextStyle.

- Клієнт (Form1) зберігає зовнішній стан (індекси слів у тексті) і передає його легковаговикам лише у момент виконання методу Apply.

Проблеми, які вирішує патерн:

- Надмірне споживання пам'яті: Запобігає створенню тисяч ідентичних об'єктів Font та Color для кожного слова (наприклад, ключового слова "public"), що призвело б до значних витрат ресурсів.
- Продуктивність: Зменшує навантаження на збирач сміття (Garbage Collector) завдяки меншій кількості об'єктів у купі.

Переваги використання:

- Економія пам'яті: Спільне використання об'єктів дозволяє працювати з великою кількістю дрібних елементів (слів, символів) без критичного зростання витрат RAM.
- Централізація: Управління стилями відбувається в одному місці (Factory), що спрощує підтримку коду.

Код:

```
1      using System.Windows.Forms;
2
3      namespace TextEditor.Flyweight
4      {
5          public interface ITextStyle
6          {
7              void Apply(RichTextBox control, int index, int length);
8          }
9      }
```

Рис. 3.1 – ITextStyle.cs

```

1  using System.Drawing;
2  using System.Windows.Forms;
3
4  namespace TextEditor.Flyweight
5  {
6      public class SharedTextStyle : ITextStyle
7      {
8          private readonly Font _font;
9          private readonly Color _color;
10
11         public SharedTextStyle(string fontFamily, float size, FontStyle style, Color color)
12         {
13             _font = new Font(fontFamily, size, style);
14             _color = color;
15         }
16
17         public void Apply(RichTextBox control, int index, int length)
18         {
19             control.Select(index, length);
20             control.SelectionFont = _font;
21             control.SelectionColor = _color;
22         }
23     }
24 }

```

Рис. 3.2 – SharedTextStyle.cs

```

1  using System.Collections.Generic;
2  using System.Drawing;
3
4  namespace TextEditor.Flyweight
5  {
6      public class TextStyleFactory
7      {
8          private readonly Dictionary<string, ITextStyle> _styles = new Dictionary<string, ITextStyle>();
9
10         public int TotalObjectsCreated => _styles.Count;
11
12         public ITextStyle GetStyle(string fontFamily, float size, FontStyle style, Color color)
13         {
14             string key = $"{fontFamily}_{size}_{style}_{color.Name}";
15
16             if (!_styles.ContainsKey(key))
17             {
18                 _styles[key] = new SharedTextStyle(fontFamily, size, style, color);
19             }
20
21             return _styles[key];
22         }
23     }
24 }

```

Рис. 3.3 – LogReportGenerator.cs

Висновок: У ході роботи було створено проєкт текстового редактора з використанням шаблону Flyweight (Легковаговик). Реалізовано основні класи: ITextStyle - інтерфейс для визначення поведінки стилів; SharedTextStyle - конкретний легковаговик, що зберігає спільний внутрішній стан (шрифт та колір); TextStyleFactory - фабрика, що керує пулом об'єктів та забезпечує їх повторне використання. Застосування патерну Flyweight дозволило досягти значної економії оперативної пам'яті при форматуванні тексту,

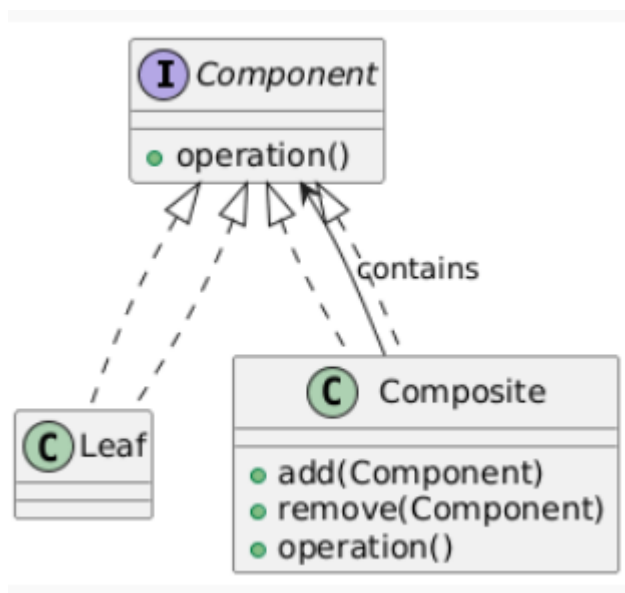
уникнувши створення дублікатів об'єктів для однакових стилів, та оптимізувати роботу програми при великій кількості елементів.

Відповіді на контрольні питання:

1. Яке призначення шаблону «Композит»?

Шаблон Композит (Composite) використовується для представлення об'єктів у вигляді ієрархічної деревоподібної структури, де окремі об'єкти та групи об'єктів обробляються однаково. Це дозволяє клієнту працювати з одиничними елементами та колекціями через спільний інтерфейс.

2. Нарисуйте структуру шаблону «Композит».



3. Які класи входять в шаблон «Композит», та яка між ними взаємодія?

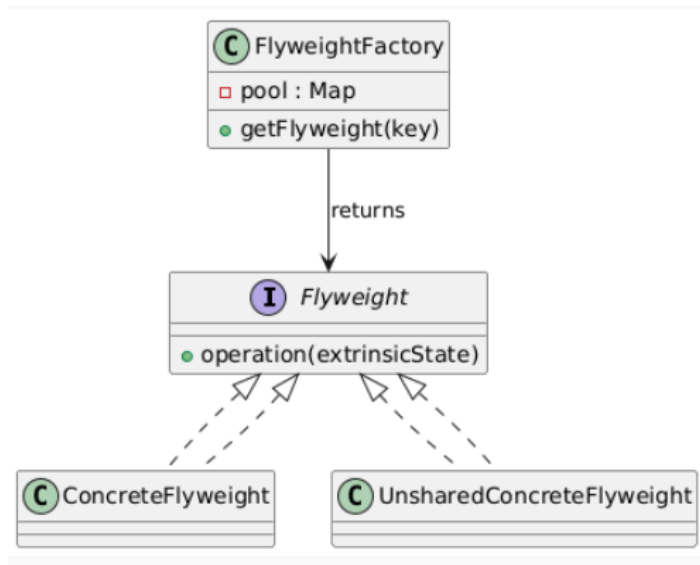
- **Component** - спільний інтерфейс для листків і контейнерів
- **Leaf** - елементарний об'єкт без дочірніх елементів
- **Composite** - вузол, який містить інші компоненти

Composite містить список **Component** і делегує виклики дочірнім елементам.

4. Яке призначення шаблону «Легковаговик»?

Шаблон Легковаговик дозволяє економити пам'ять шляхом повторного використання спільних об'єктів, розділяючи стан на внутрішній (shared) і зовнішній (у клієнта). Використовується при великій кількості дрібних однотипних об'єктів.

5. Нарисуйте структуру шаблону «Легковаговик».



6. Які класи входять в шаблон «Легковаговик», та яка між ними взаємодія?

- Flyweight - інтерфейс об'єктів з внутрішнім станом
- ConcreteFlyweight - спільний об'єкт
- UnsharedFlyweight - об'єкт, який не розділяється
- FlyweightFactory - повертає існуючі екземпляри або створює нові

Зовнішній стан передається в методи замість зберігання в об'єкті.

7. Яке призначення шаблону «Інтерпретатор»?

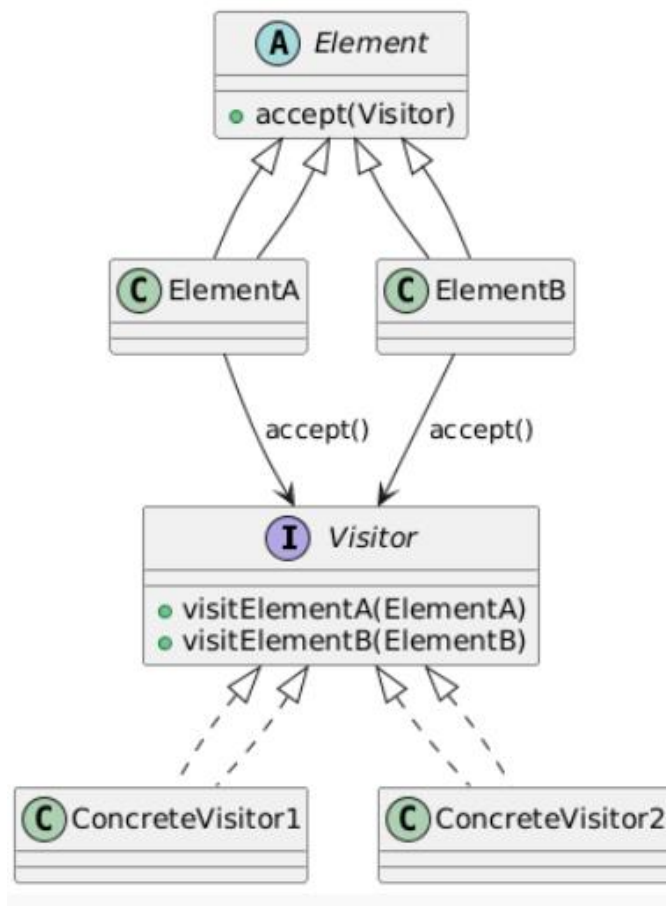
Шаблон Інтерпретатор визначає спосіб опису граматики та інтерпретації

мовоподібних команд. Використовується для побудови простих мов, обробки виразів, формул, логічних правил.

8. Яке призначення шаблону «Відвідувач»?

Шаблон Відвідувач (Visitor) дозволяє додавати нові операції до об'єктів без модифікації їхніх класів, переміщуючи поведінку в окремий клас-відвідувач. Це корисно, коли треба виконувати багато різних операцій над об'єктами складної структури.

9. Нарисуйте структуру шаблону «Відвідувач».



10. Які класи входять в шаблон «Відвідувач», та яка між ними взаємодія?

- **Element** - елемент структури
- **ConcreteElement** - конкретні елементи
- **Visitor** - інтерфейс операцій

- ConcreteVisitor — реалізації операцій

Взаємодія через double dispatch:

`element.accept(visitor) → visitor.visit(element).`